

1. Node.js là gì?

- **Định nghĩa:** Node.js là một **môi trường chạy mã JavaScript (runtime environment)** đa nền tảng, mã nguồn mở. Nó cho phép bạn chạy mã JavaScript ở phía máy chủ (backend) thay vì chỉ chạy trên trình duyệt (frontend).
- **Lõi công nghệ:** Node.js được xây dựng dựa trên **V8 JavaScript Engine** (cỗ máy cốt lõi của Google Chrome), giúp biên dịch mã JavaScript trực tiếp thành mã máy (machine code) với tốc độ cực kỳ nhanh.

2. Đặc điểm cốt lõi của Node.js

- **Bất đồng bộ và Không chặn (Asynchronous & Non-blocking I/O):** Trong các ngôn ngữ truyền thống (như PHP, Java), khi có một yêu cầu đọc file hoặc truy vấn database, máy chủ sẽ dừng lại chờ tác vụ đó xong mới làm tiếp. Node.js thì khác, nó sẽ "gửi" tác vụ đó đi xử lý ngầm và ngay lập tức quay lại nhận yêu cầu tiếp theo. Khi tác vụ ngầm hoàn thành, nó sẽ báo lại (thông qua callback/promise).
- **Đơn luồng (Single-threaded) nhưng khả năng mở rộng cao:** Node.js sử dụng một luồng duy nhất kết hợp với **Event Loop** (Vòng lặp sự kiện) để quản lý hàng ngàn kết nối đồng thời mà không bị quá tải bộ nhớ.
- **Hệ sinh thái khổng lồ (NPM):** Node Package Manager (NPM) là kho lưu trữ thư viện mã nguồn mở lớn nhất thế giới. Bạn có thể tìm thấy các gói (packages) cho gần như mọi tính năng bạn cần, giúp tiết kiệm hàng trăm giờ code từ con số 0.

3. Node.js thường được dùng để làm gì?

Nhờ đặc tính nhanh và xử lý nhiều luồng dữ liệu cùng lúc tốt, Node.js cực kỳ phù hợp cho:

- **Ứng dụng thời gian thực (Real-time Applications):** Ứng dụng chat (Zalo, Messenger), ứng dụng giao dịch chứng khoán, game online nhiều người chơi.
- **API (Giao diện lập trình ứng dụng):** Xây dựng RESTful API hoặc GraphQL cho các ứng dụng web/mobile (Single Page Applications - SPAs).
- **Dịch vụ Streaming (Phát trực tuyến):** Xử lý luồng dữ liệu liên tục như video, âm thanh (Netflix cũng sử dụng Node.js).

- **Kiến trúc Microservices:** Chia nhỏ ứng dụng lớn thành các dịch vụ nhỏ, độc lập.

4. Ưu điểm và Nhược điểm

Điểm mạnh	Điểm yếu
- "Full-stack" bằng 1 ngôn ngữ: Dùng chung JavaScript cho cả Frontend và Backend, giúp team dễ giao tiếp và tái sử dụng code.	- Không hợp với tác vụ nặng về CPU: Vì là đơn luồng, nếu có một tác vụ tính toán toán học quá phức tạp (render video, AI), nó sẽ "chặn" toàn bộ hệ thống.
- Tốc độ thực thi cực nhanh nhờ V8 Engine và Non-blocking I/O.	- Callback Hell: Nếu không biết cách viết code (dùng Async/Await hay Promise), code sẽ bị lồng nhau rất sâu, khó đọc và khó bảo trì.
- Cộng đồng hỗ trợ khổng lồ.	

5. Các Framework phổ biến nhất của Node.js

Dù bạn có thể viết Backend bằng Node.js thuần, nhưng để nhanh và chuẩn hóa hơn, người ta thường dùng các framework:

- **Express.js:** Framework phổ biến nhất, nhẹ, linh hoạt và tối giản. (Rất tốt cho người mới bắt đầu).
- **NestJS:** Framework hiện đại, sử dụng TypeScript, có cấu trúc chặt chẽ (tương tự Angular/Spring Boot). Phù hợp cho các dự án quy mô lớn (Enterprise).
- **Koa.js:** Được tạo ra bởi đội ngũ của Express, nhỏ gọn hơn và hiện đại hơn.